

DevTrack - Auditoria de Segurança e Arquitetura

Relatório direcionador para correção de falhas no front-end, back-end, autenticação, comunicação entre camadas e sustentabilidade do código.

Data da auditoria: 20/05/2026. Escopo analisado: React + Tailwind + Vite, Java + Spring Boot, JWT, rotas privadas, perfil de usuário e comunicação via Axios.

Este documento é um guia técnico. A palavra final sempre é do dono do projeto. Nenhuma correção deve ser aplicada sem validação de prioridade, impacto e momento do MVP.

Segurança: 5/10

MVP funcional, mas ainda com riscos altos antes de produção.

Sustentabilidade: 6/10

Base simples e compreensível, porém com duplicações e padrões ainda soltos.

1. Resumo Executivo

O DevTrack está bem encaminhado para um MVP local: usa DTOs, BCrypt, JWT stateless, rotas privadas no backend e interceptor Axios no frontend. Os riscos mais urgentes são vazamento de secrets, logs sensíveis, token no localStorage, ausência de rate limit no login, validações divergentes e ausência de tratamento global de erros.

2. Mapa do Fluxo Auth Atual

ETAPA	FLUXO ATUAL	ARQUIVOS PRINCIPAIS	RISCO OBSERVADO
Registro	Front valida e envia <code>POST /user/register</code> . Backend salva senha com BCrypt.	<code>Register.jsx</code> , <code>UserController.java</code> , <code>UserService.java</code> , <code>UserRequestDTO.java</code>	Senha forte só é exigida no front; backend exige apenas tamanho.
Login	Front envia email e senha para <code>POST /user/login</code> . Backend retorna usuário e token.	<code>Login.jsx</code> , <code>LoginRequestDTO.java</code> , <code>LoginResponseDTO.java</code>	Logs imprimem senha, hash e token. DTO de login não usa validação Jakarta.
Sessão	Token e usuário são salvos em <code>localStorage</code> .	<code>tokenService.js</code> , <code>api.js</code>	XSS futuro pode roubar token e dados locais.
Rotas privadas	<code>PrivateRoute</code> checka apenas se existe token.	<code>PrivateRoute.jsx</code> , <code>AppRoutes.jsx</code>	Token expirado ou inválido ainda abre UI privada até a primeira falha de API.
Perfil	Update usa email do <code>Principal</code> . Senha e delete pedem senha atual.	<code>ProfileEdit22.jsx</code> , <code>Dashboard22.jsx</code> , <code>UserService.java</code>	JWT usa email como subject; mudança de email invalida a identidade antiga.

3. Erros e Riscos do Back-end

GRAVIDADE	ARQUIVO / CLASSE	PROBLEMA	COMO PODE SER EXPLORADO OU VIRAR BUG	MELHOR CORREÇÃO	OUTRAS OPÇÕES	IMPACTO	PRIORIDADE
Crítica	<code>application.properties</code>	Senha do banco e <code>jwt.secret</code> estão no repositório.	Se o repo vazar, banco e tokens ficam comprometidos.	Usar variáveis de ambiente e remover secrets do histórico quando possível.	Usar <code>.env</code> local ignorado pelo git e profiles Spring.	Alto positivo; exige ajustar ambiente local.	Agora
Crítica	<code>UserService.java</code>	<code>System.out.println</code> imprime email, senha, hash e token.	Logs viram vazamento completo de credenciais e sessão.	Remover logs sensíveis imediatamente.	Usar logger com mascaramento e nível debug só para dados não sensíveis.	Alto positivo; sem quebra funcional.	Agora
Alta	<code>UserService.java</code> , <code>UserController.java</code>	Login sem rate limit ou bloqueio progressivo.	Permite brute force contra contas reais.	Adicionar rate limit por IP e por email.	Bucket4j local no MVP; Redis ou gateway no futuro.	Reduz ataque automatizado; cuidado para não bloquear usuário legítimo.	Agora
Alta	<code>LoginRequestDTO.java</code> , <code>UserController.java</code>	DTO de login não tem <code>@NotBlank</code> , <code>@Email</code> ou <code>@Size</code> , e endpoint não usa <code>@Valid</code> .	Campos nulos ou muito grandes podem gerar erro genérico e carga desnecessária.	Adicionar validações Jakarta e <code>@Valid</code> .	Validar manualmente no service, embora seja menos limpo.	Melhora segurança e UX; baixo risco.	Agora
Alta	<code>UserRequestDTO.java</code> , <code>UserPasswordUpdateRequestDTO.java</code>	Complexidade de senha forte não é exigida no backend.	Atacante pode chamar API direto e criar senha fraca.	Criar validador backend para maiúscula, minúscula, número e símbolo.	Usar <code>@Pattern</code> com mensagem clara.	Alinha API com regra de negócio; baixo risco.	Agora
Alta	Ausente: <code>@ControllerAdvice</code>	Não há tratamento global de exceções e respostas de erro padronizadas.	Erros de validação, duplicidade e runtime podem virar 500 ou mensagens inconsistentes.	Criar handler global com <code>{ message, fieldErrors }</code> .	Tratar por controller, mas isso escala mal.	Melhora API e front; exige ajustar mensagens esperadas.	Agora
Média	<code>JwtService.java</code>	JWT usa email como subject.	Ao mudar email, token antigo perde coerência e pode gerar logout ou falhas confusas.	Usar <code>userId</code> como subject e email como claim opcional.	Gerar novo token quando email mudar.	Médio; precisa ajustar filtro e service.	Depois do MVP
Média	<code>UserService.java</code> , <code>UserRepository.java</code>	Update de email/username depende da constraint do banco para duplicidade.	Usuário recebe erro ruim; pode expor stack ou quebrar UX.	Checar existência antes de salvar e retornar mensagem amigável.	Tratar <code>DataIntegrityViolationException</code> no handler global.	Melhora UX; baixo risco.	Agora
Média	<code>SecurityConfig.java</code> , <code>CorsConfig.java</code> , <code>UserController.java</code>	CORS está espalhado entre config global e <code>@CrossOrigin</code> .	Ambientes podem divergir e liberar origem errada sem perceber.	Manter CORS apenas em <code>CorsConfig</code> .	Profiles por ambiente.	Organização melhor; baixo risco.	Depois do MVP
Média	<code>application.properties</code>	<code>ddl-auto=update</code> e <code>show-sql=true</code> .	Migração imprevisível e logs verbosos em ambiente real.	Usar profiles; dev pode manter update, prod deve usar validate/migrations.	Flyway ou Liquibase quando o schema estabilizar.	Evita surpresa em produção.	Depois do MVP
Baixa	<code>User.java</code>	<code>createdAt</code> existe, mas não é preenchido automaticamente.	Perfil pode exibir data indisponível.	Adicionar <code>@CreationTimestamp</code> ou setar no service.	Usar auditoria JPA no futuro.	Melhora consistência de dados.	Depois do MVP

4. Erros e Riscos do Front-end

GRAVIDADE	ARQUIVO / COMPONENTE	PROBLEMA	COMO PODE SER EXPLORADO OU VIRAR BUG	MELHOR CORREÇÃO	OUTRAS OPÇÕES	IMPACTO	PRIORIDADE
Alta	<code>tokenService.js</code>	JWT e usuário ficam em <code>localStorage</code> .	Qualquer XSS futuro pode roubar token e assumir a conta.	Em produção, migrar para cookie <code>HttpOnly Secure SameSite</code> .	No MVP, manter <code>localStorage</code> com CSP forte e sem HTML dinâmico perigoso.	Alto; exige ajuste de backend se mudar para cookies.	Agora antes de produção
Média	<code>AppRoutes.jsx</code>	Rota <code>/dashboard</code> está pública.	Usuário não logado acessa uma tela privada/demo.	Proteger com <code>PrivateRoute</code> ou remover rota antiga.	Redirecionar para <code>/dashboard22</code> .	Baixo risco de quebra; melhora clareza.	Agora
Média	<code>PrivateRoute.jsx</code>	Guard só verifica se existe token, não validade ou expiração.	Token expirado mantém UI privada até uma request falhar.	Decodificar expiração ou carregar <code>/user/me</code> .	Logout global no primeiro 401 com redirect.	Melhora UX e segurança percebida.	Agora
Média	<code>api.js</code>	Interceptor remove só token no 401, não remove usuário nem redireciona.	<code>LocalStorage</code> fica inconsistente e a UI pode exibir usuário antigo.	Chamar <code>logout()</code> e disparar redirecionamento.	Event bus ou callback de sessão expirada.	Médio; precisa evitar loop de navegação.	Agora
Média	<code>validator.js</code> , <code>Register.jsx</code> , DTOs backend	Validações de username/senha divergem entre front e back.	Front aceita algo que o backend rejeita, ou API aceita algo que o front impediria.	Alinhar regex, tamanhos e mensagens com contrato único.	Documentar contrato de API em uma tabela simples.	Reduz bugs de formulário.	Agora
Baixa	<code>validator.js</code>	Função <code>validateLogin</code> morta usa variáveis inexistentes.	Lint quebra e manutenção fica confusa.	Remover função ou mover lógica real para <code>Login.jsx</code> .	Exportar função pura recebendo dados como parâmetro.	Baixo risco; melhora qualidade.	Agora
Baixa	<code>ProfileEdit22.jsx</code>	Há <code>console.log</code> com dados de atualização do usuário.	Dados ficam no console do navegador em uso real.	Remover logs.	Logger dev-only condicionado por ambiente.	Baixo risco; sem quebra funcional.	Agora
Baixa	<code>Profile.jsx</code>	Componente <code>GitHubIcon</code> é criado dentro do render.	ESLint acusa risco de reset de estado e padrão ruim.	Mover componente para fora de <code>Profile</code> .	Usar ícone de biblioteca se existir.	Baixo; corrige lint.	Agora

5. Planejamento de Correção Escalável

Ordem sugerida: começar por correções pequenas e sem quebra, depois fechar riscos críticos, depois padronizar arquitetura. A palavra final sobre a ordem sempre é sua.

ETAPA	OBJETIVO	ARQUIVOS	CAMINHO SUGERIDO	VALIDAÇÃO
1	Limpeza sem risco funcional	validator.js , Profile.jsx , ProfileEdit22.jsx	Remover função morta, mover <code>GitHubIcon</code> , remover logs do front.	<code>npm run lint</code> deve reduzir erros atuais.
2	Fechar vazamento crítico no backend	UserService.java	Remover todos os <code>System.out.println</code> de senha, hash, email e token.	Login continua funcionando; logs não exibem segredo.
3	Externalizar secrets sem quebrar dev local	application.properties	Trocar valores por placeholders como <code>{DB_PASSWORD}</code> e documentar env vars.	Subir backend local com variáveis configuradas.
4	Padronizar erros da API	Novo <code>GlobalExceptionHandler</code> , DTO de erro	Criar handler para validação, runtime e duplicidade de banco.	Front recebe sempre <code>message</code> e opcionalmente <code>fieldErrors</code> .
5	Alinhar validações de login e cadastro	LoginRequestDTO.java , UserRequestDTO.java , UserPasswordUpdateRequestDTO.java , validator.js	Adicionar constraints no login e regra forte de senha no backend.	Requests inválidas retornam 400 controlado.
6	Proteger navegação e sessão no front	AppRoutes.jsx , PrivateRoute.jsx , api.js , tokenService.js	Proteger/remover <code>/dashboard</code> , limpar usuário no 401 e redirecionar.	Token inválido ou expirado derruba sessão de forma previsível.
7	Reduzir brute force	UserController.java , UserService.java , possível config de rate limit	Adicionar limite simples por IP/email para login.	Tentativas repetidas recebem 429 sem bloquear fluxo normal.
8	Melhorar identidade do JWT	JwtService.java , JwtAuthenticationFilter.java , UserService.java	Mudar subject para <code>userId</code> e buscar usuário por id quando necessário.	Mudança de email não quebra identidade do token novo.
9	Criar fonte de verdade para usuário autenticado	Novo endpoint <code>/user/me</code> , <code>userService.js</code>	Front deixa de confiar apenas no usuário salvo em localStorage.	Reload da página busca dados atuais do banco.
10	Preparar produção sem exagero enterprise	application.properties , profiles Spring, documentação	Separar dev/prod, desligar <code>show-sql</code> , planejar migrations.	Ambiente local continua simples; produção fica mais segura.

6. Checklist por Prioridade

Agora

ITEM	PARTE	ARQUIVOS
Remover logs sensíveis	Back-end	UserService.java
Remover secrets do arquivo de config	Back-end	application.properties
Adicionar validação no login	Back-end	LoginRequestDTO.java , UserController.java
Repetir validação forte de senha no backend	Back-end	UserRequestDTO.java , UserPasswordUpdateRequestDTO.java
Criar handler global de erros	Back-end	Novo pacote/classe de exception
Proteger/remover rota <code>/dashboard</code>	Front-end	AppRoutes.jsx
Corrigir limpeza de sessão no 401	Front-end	api.js , tokenService.js
Corrigir lint e função morta	Front-end	validator.js , Profile.jsx

Depois do MVP

ITEM	PARTE	ARQUIVOS
Criar <code>/user/me</code>	Ambos	UserController.java , UserService.java , userService.js
Usar user id no JWT	Back-end	JwtService.java , JwtAuthenticationFilter.java
Unificar componentes antigos	Front-end	Dashboard.jsx , Dashboard22.jsx , ProfileEdit.jsx , ProfileEdit22.jsx
Profiles dev/prod	Back-end	application.properties e configs por ambiente

Futuro

ITEM	MOTIVO
Refresh token ou rotação de sessão	Melhor UX e controle de sessão.
Cookies HttpOnly em produção	Reduz impacto de XSS roubando token.
CSP e headers de segurança	Endurece front-end contra XSS e misuse.
Roles e permissões	Necessário quando houver admin, planos, times ou recursos compartilhados.
Flyway/Liquibase	Evita schema drift conforme o banco crescer.

7. O Que Está Bem Feito

ÁREA	PONTO POSITIVO	POR QUE MANTER
Back-end	Spring Security protege qualquer rota não liberada explicitamente.	É uma boa postura padrão: privado por padrão, público por exceção.
Back-end	Senha é salva com BCrypt.	É a escolha correta para senhas no MVP.
Back-end	Uso de DTOs em vez de expor entity diretamente.	Ajuda a controlar dados expostos e contratos de API.
Ambos	Update de senha e delete exigem senha atual.	Reduz dano caso alguém use uma sessão aberta indevidamente.
Front-end	Axios centralizado com interceptor de Authorization.	Evita repetir header e facilita evolução futura.
Arquitetura	Projeto ainda está pequeno e legível.	Correções estruturais agora são baratas e evitam dívida grande.

8. Plano Sem Overengineering

Não é necessário criar arquitetura enterprise agora. O caminho profissional para um MVP SaaS real é fechar vazamentos, padronizar erros, alinhar validações, melhorar sessão e só depois introduzir recursos como refresh token, roles e migrations formais.

A sequência ideal é: segurança básica forte primeiro, consistência de API depois, e arquitetura incremental conforme surgirem recursos reais. O projeto não precisa de microserviços, RBAC completo ou camadas complexas no momento. Precisa de padrões simples que possam ser repetidos sem bagunçar o crescimento.